

Probabilidad Y Estadística II

Memoria

Grupo 1

Integrantes:

- Haodi Lin Sun
- Jam Rian Raguine Rabang
- Felipe Agra Domínguez
- Diana Moslemi Baghermanesh

Índice:

1. Ajuste de Modelo y Muestreo

- 1.1. Ajuste de Modelo
- 1.2. Muestreo

2. Estimación Clásica de Máxima Verosimilitud

- 2.1. Estimación puntual
- 2.2. Estimación por intervalos

3. Estimación Bayesiana

4. Contrastes

- 4.1. Contrastes Paramétricos
- 4.2. Contrastes No Paramétricos

5. Regresión lineal

6. Anexo: Conjunto de librerías utilizadas

1. Ajuste de modelo y muestreo

Def: El ajuste de modelo es el proceso de encontrar un modelo matemático que describa adecuadamente un conjunto de datos. Y el muestreo es otra técnica utilizada para seleccionar un subconjunto representativo de datos en una población.

Enunciados:

Ajuste de Modelo.

Dada la muestra de datos de la variable Data.Carbohydrate:
Sample1 (0.691666, 0, 0.3280065, 0.6374219, 0.6126026, 0, 0.7169533, 0.6157937, 0.690012, 0.5554588, 0.6219643, 0.7177884, 0.6931629, 0.5971366, 0.629137, 0.7084673, 0, 0.7186016, 0.5481317, 0.7437346);

Responder a las preguntas siguientes realizando las inferencias.
Explicar el procedimiento e incluir código R y comentarios al resultado en la memoria.

```
library(e1071);library(MASS).
```

Muestreo.

Dadas las medias muestrales de 30 muestras aleatorias de la variable Data.Major.Minerals.Magnesium:

Medias_Muestrales_30 (1.900526, 1.967167, 2.179823, 1.533017, 2.19935, 1.83783, 2.031163, 1.944047, 2.240126, 1.95977, 1.918929, 1.772965, 2.084756, 2.253169, 1.90685, 1.769868, 2.105862, 1.884825, 1.915734, 2.035991, 1.97289, 2.365661, 1.877642, 1.900701, 1.682702, 1.825557, 2.37747, 2.255502, 1.626245, 1.895156);

Responder a las preguntas siguientes realizando las inferencias.
Explicar el procedimiento e incluir código R y comentarios al resultado en la memoria.

```
library(e1071);library(MASS);library(fitdistrplus).
```

Muestreo

Dadas las proporciones muestrales 30 muestras de la variable
Data.Sugar.Total==S1:

Proporciones_Muestrales_30 (0.5, 0.7, 0.5, 0.6, 0.65, 0.6, 0.55,
0.55, 0.65, 0.55, 0.5, 0.5, 0.7, 0.35, 0.6, 0.55, 0.55, 0.55, 0.65, 0.8,
0.7, 0.65, 0.7, 0.6, 0.65, 0.55, 0.65, 0.6, 0.65, 0.5);

Responder a las preguntas siguientes realizando las inferencias.

Explicar el procedimiento e incluir código R y comentarios al
resultado en la memoria.

1.1 Ajuste de Modelo

```
> summary(Sample1)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.0000	0.5536	0.6256	0.5413	0.6970	0.7437

Para ajustar el modelo, tenemos que analizar la muestra
Sample1, extrayendo de esta el mínimo, primer cuartil,
mediana, media, tercer cuartil y el máximo, con la función
Summary();

También tenemos que calcular la asimetría con la función
skewness(); y curtosis o coeficiente de apuntamiento.

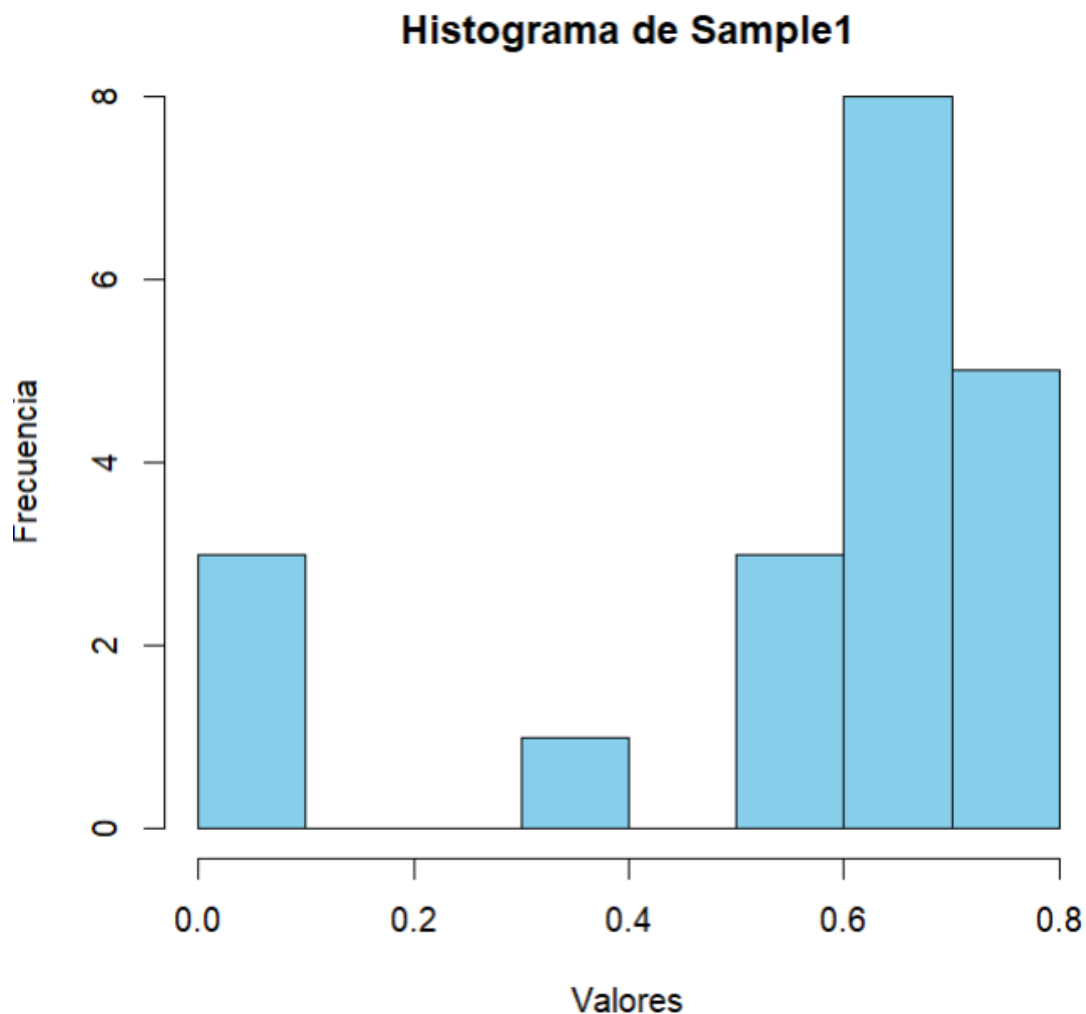
```
> skewness(Sample1)
```

```
[1] -1.408825
```

```
> kurtosis(Sample1)
```

[1] 0.3884451

```
hist(Sample1, main = "Histograma de Sample1", xlab =  
"Valores", ylab = "Frecuencia", col = "skyblue", border =  
"black")
```

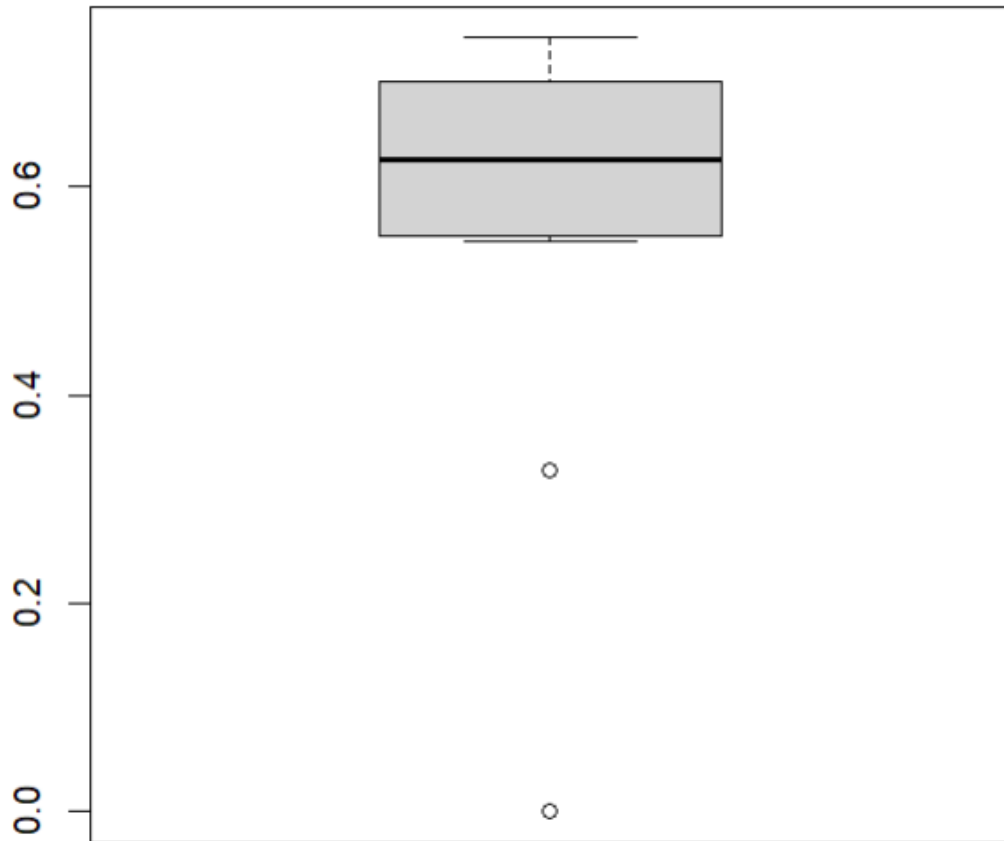


El histograma que hemos creado divide nuestros datos en 8 clases de intervalos, donde se aprecia la cúspide de los datos entre el 0.6 y el 0.7, y el mínimo se aprecia entre los huecos que se ha dejado entre 0.1 y 0.3, y entre 0.4 y 0.5.

Como la asimetría calculada es negativa, apreciamos a la vez en el gráfico una asimetría a la izquierda. La media que se calcula posteriormente es 0.54130196, y los datos que se

muestran están un poco cerca de esta, por lo que tenemos un valor ligeramente positivo de curtosis.

```
>boxplot(Sample1)
```



En el boxplot creado con anterioridad, se muestra la mediana con la línea horizontal de dentro de la caja. Se pueden observar valores atípicos que se extienden a la caja.

Ahora es cuando empezamos a ajustar los datos:

Primero se calcula el estimador de la media y desviación típica con la función `fitdistr`:

```
> ajuste <- fitdistr(Sample1, densfun = "normal")
> ajuste
      mean      sd
0.54130196 0.24414353
```

Y utilizamos `kolmogorov-smirnov` para observar el ajuste

```
out <- ks.test(Sample1, "pnorm", mean(Sample1),
sd(Sample1))
print(out)
```

Asymptotic one-sample Kolmogorov-Smirnov test

data: Sample1

D = 0.31088, p-value = 0.04189

alternative hypothesis: two-sided

Como el p-value es positivo mayor que 0.1, podemos decir que el ajuste es bueno.

Ahora lo probamos ajustando los datos con una distribución exponencial. Calculamos el estimador λ .

```
> ajuste2 <- fitdistr(Sample1,"exponential")
> ajuste2
```

1.8473977

(0.4130907)

Y utilizamos el test de kolmogorov-smirnov

```
ks_exp <- ks.test(Sample1, "pexp", ajustado$estimate[1],  
ajustado$estimate[2])
```

```
ks_exp$p.value
```

```
[1] 0.0009718761
```

Pero el ajuste es menor que 0.1, con lo cual, el ajuste no es bueno.

1.2 Muestreo

Para su representación, podemos calcular el estimador de la media y de la desviación típica:

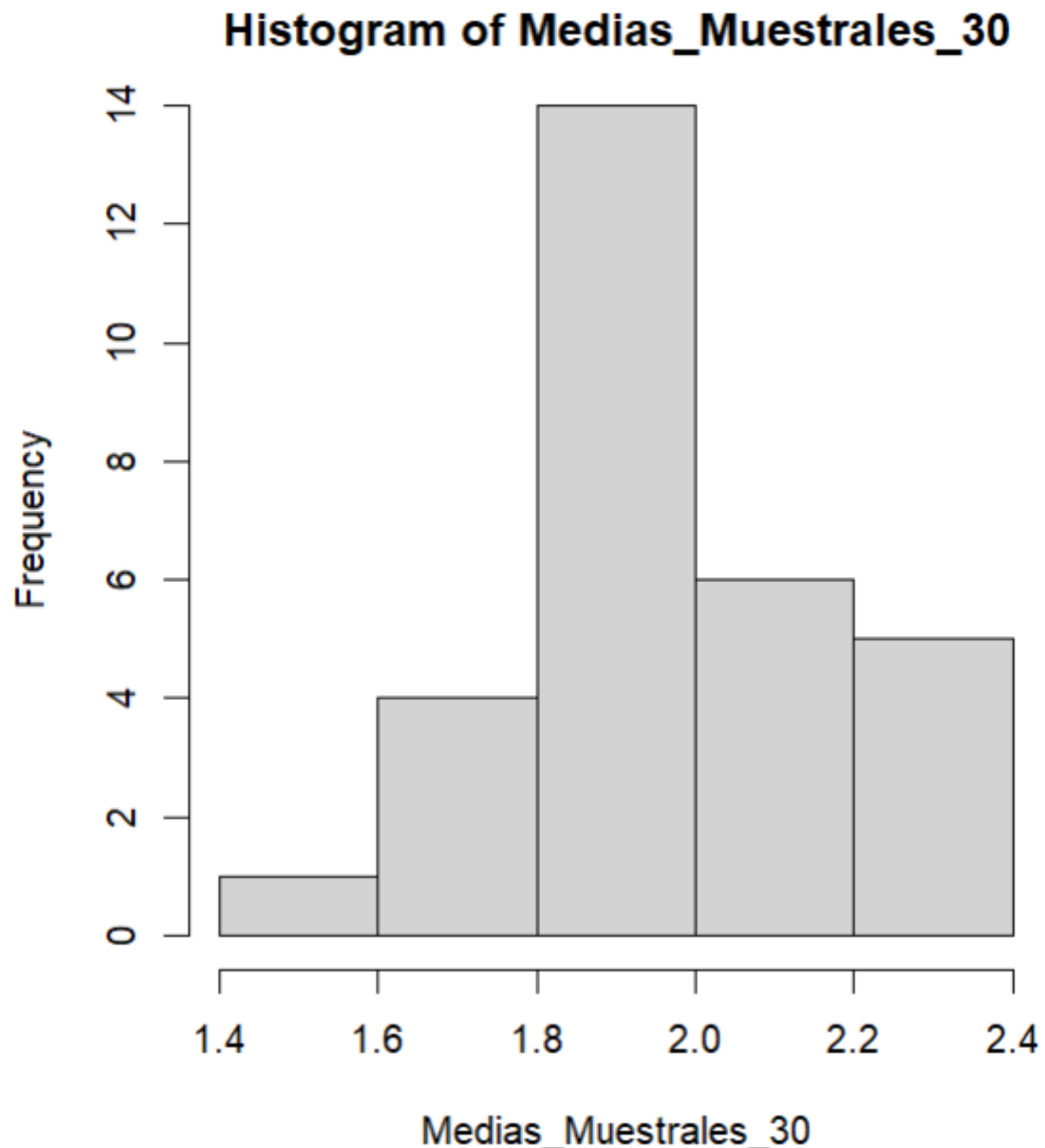
```
> mean(Medias_Muestrales_30)
```

```
[1] 1.974043
```

```
> sd(Medias_Muestrales_30)
```

```
[1] 0.2070586
```

A su vez, representarlo en un histograma:



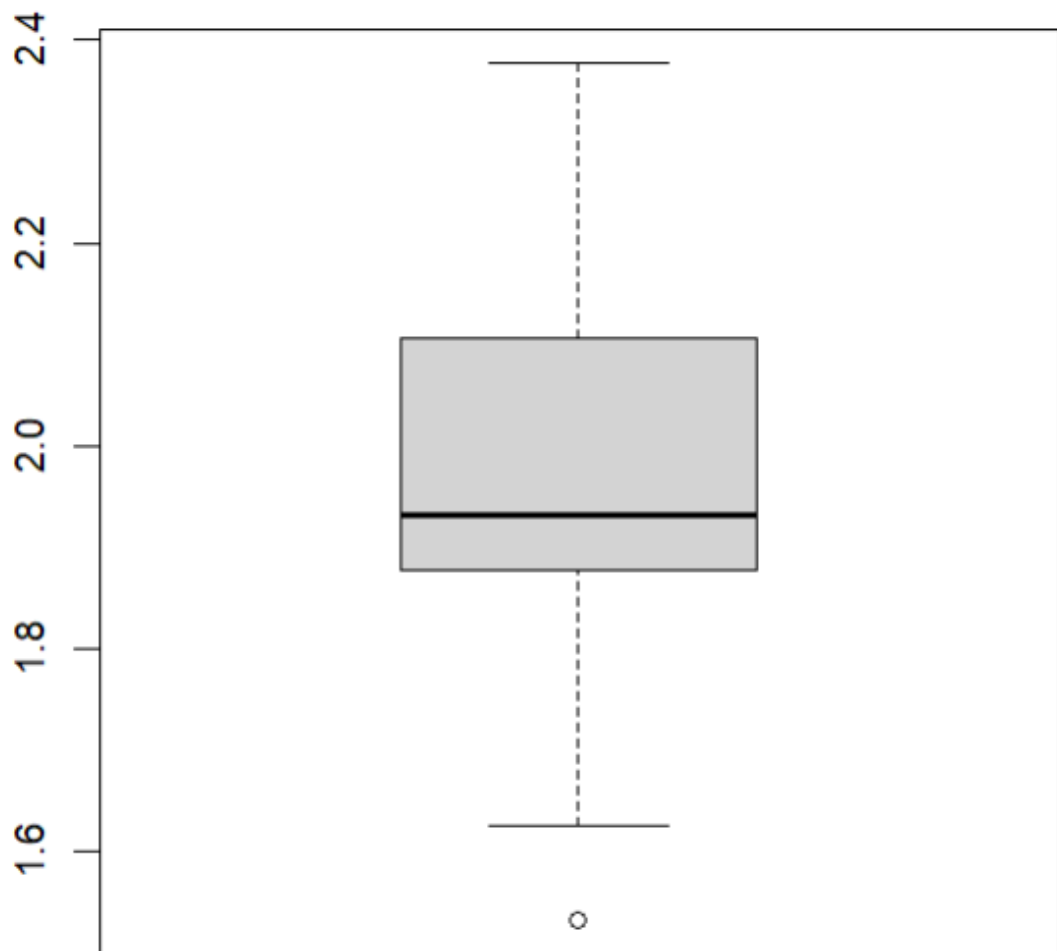
El histograma representado cuenta con 5 intervalos con una tendencia hacia el centro, por eso, la asimetría se acerca a 0 o 0.1:

```
> skewness(Medias_Muestrales_30)
```

```
[1] 0.1284655
```

Como es positivo, la distribución de datos se extiende hacia la derecha. El máximo se encuentra en el intervalo 1.8-2.0, y el mínimo en el intervalo 1.4-1.6

También podemos representarlo con un boxplot:



En el que podemos apreciar una mediana que está cerca de la media. No hay mucha asimetría, ya que los datos están posicionados cerca del intervalo central.

Ajustamos la muestra a la distribución normal:

```
ajuste <- fitdistr(Medias_Muestrales_30, "normal")
```

```
ajuste
```

```
ks_norm <- ks.test(Medias_Muestrales_30, "pnorm",
ajuste$estimate[1], ajuste$estimate[2])

ks_norm$p.value
> ks_norm$p.value
[1] 0.5922188
```

Y vemos que el p-value es mayor que 0.1, con lo cual el ajuste es bueno.

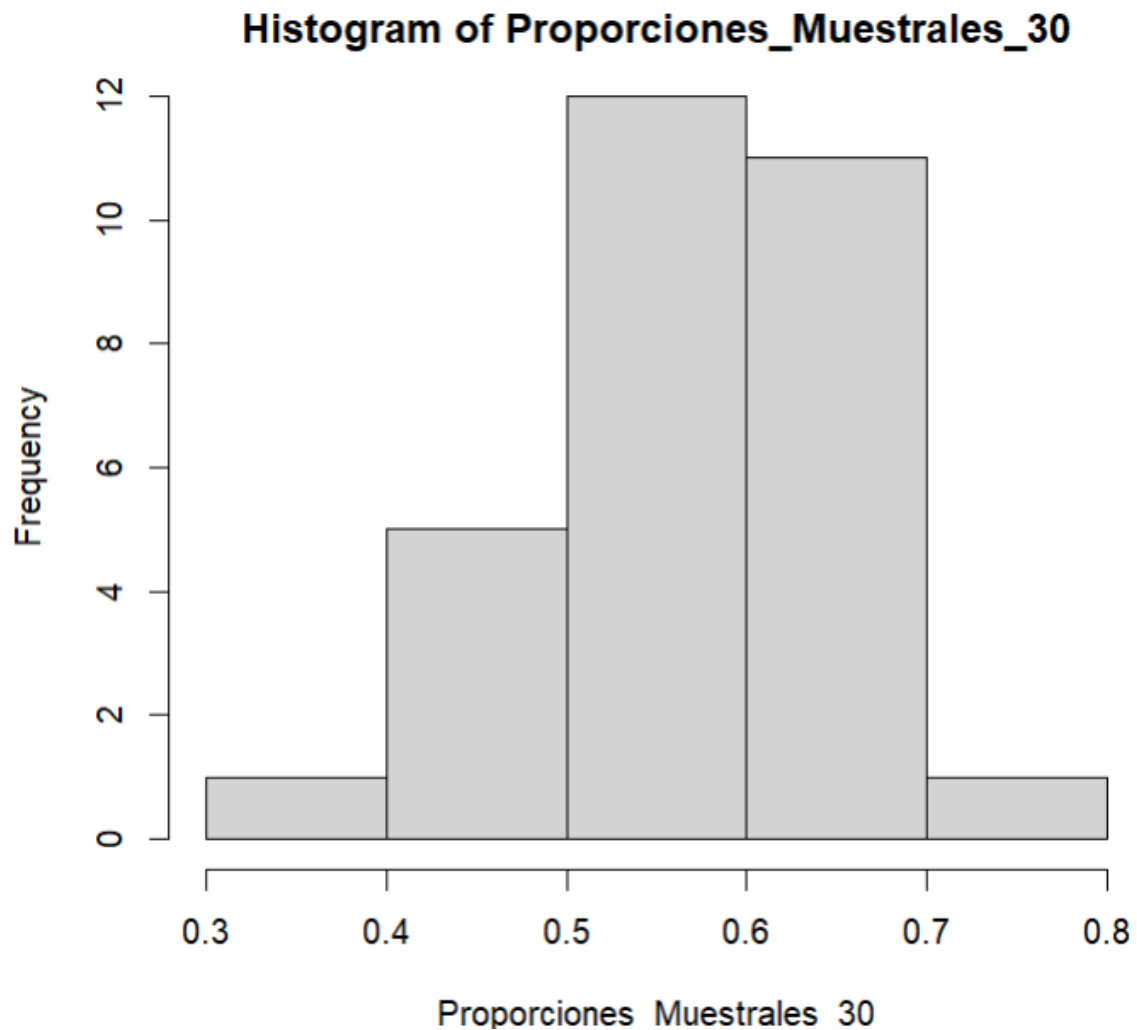
Ahora lo hacemos con las proporciones muestrales:

Antes de nada, representamos la media y la desviación típica, y analizamos el histograma:

```
> ajusteProp <- fitdistr(Proporciones_Muestrales_30,
"normal")

> ajusteProp

      mean      sd 
0.59500000 0.08693868
```



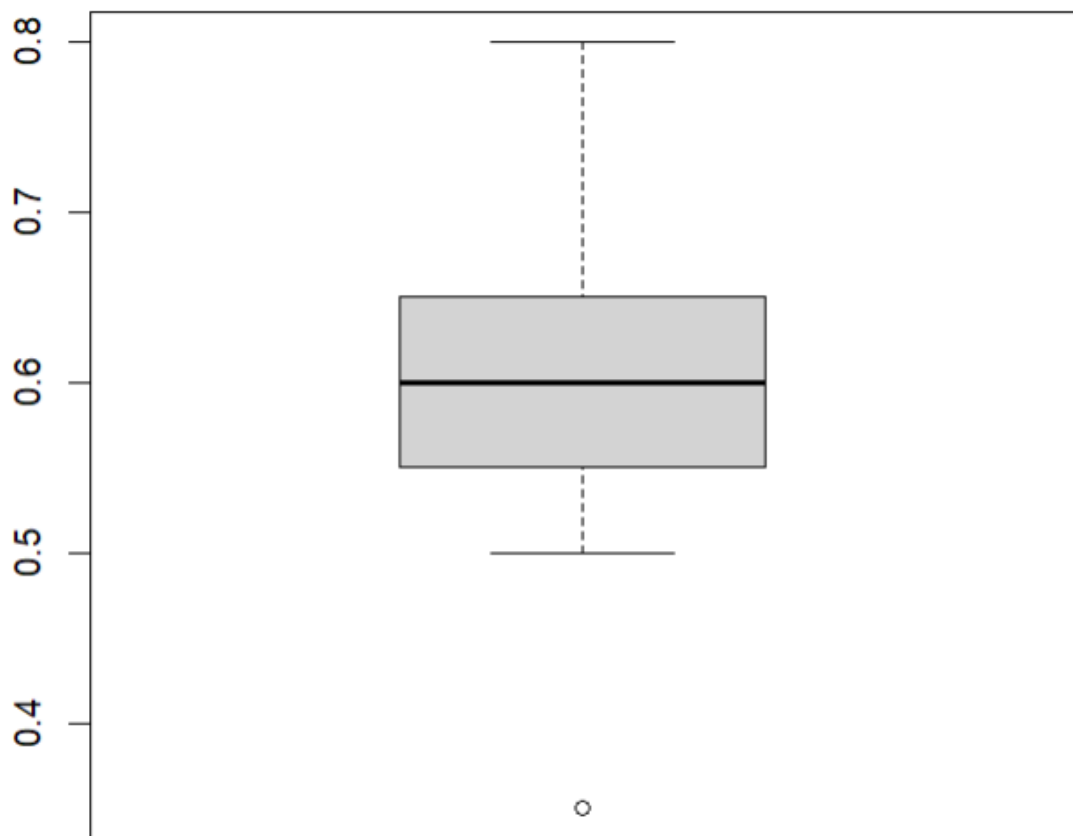
El histograma tiene 5 intervalos ligeramente asimétrico a la izquierda, ya que el indicador de asimetría es negativa:

```
> skewness(Proporciones_Muestrales_30)
```

```
[1] -0.2516664
```

El máximo está entre los intervalos 0.5- 0.6, y el mínimo se refleja en el 0.3-0.4, con valor similar al del 0.7-0.8.

La media es 0.595 y la desviación típica es 0.08693868, una variación no muy alta.



```
> boxplot(Proporciones_Muestrales_30)
```

La mediana está clavada en 0.6, pero con cuartiles 1 y 3 un poco asimétricos, donde hay más concentración de datos entre el 0.6 y el 0.7, cerca del cuartil 3, y tenemos un valor atípico a la izquierda, por lo que nos desplaza ligeramente la media.

Hemos utilizado el siguiente comando para ajustar la muestra, y con ella calculamos la media y la desviación típica.

```
> ajusteProp <- fitdistr(Proporciones_Muestrales_30,  
"normal")
```

Y sacamos el p-value:

```
ks_normP <- ks.test(Proporciones_Muestrales_30, "pnorm",  
ajusteProp$estimate[1], ajusteProp$estimate[2])
```

```
ks_normP$p.value
```

```
[1] 0.6310256
```

Y como el p-value es mayor que 0.1, podemos decir que el ajuste es bueno.

Hasta ahora, los ajustes que hemos realizado para muestreo siempre fueron de distribución normal.

2. Estimación clásica de máxima verosimilitud

Def: La estimación clásica es un método para estimar parámetros de un modelo maximizando la función de verosimilitud, donde está la probabilidad de observar los datos dados los parámetros.

Estimación puntual: Proporciona un único valor para el parámetro.

Estimación por intervalos: Construye un rango donde el parámetro esté probablemente. (Intervalo de confianza)

Enunciados:

Estimación puntual de Máxima Verosimilitud, mle.

Dada la muestra de datos de la variable

Data.Carbohydrate:

Sample1 (0.691666, 0, 0.3280065, 0.6374219, 0.6126026, 0, 0.7169533, 0.6157937, 0.690012, 0.5554588, 0.6219643, 0.7177884, 0.6931629, 0.5971366, 0.629137, 0.7084673, 0, 0.7186016, 0.5481317, 0.7437346);

Dada la muestra de datos pareada con Sample1 de la variable

Data.Sugar.Total:

Sample2 (S2, S1, S1, S1, S1, S1, S5, S1, S2, S1, S1, S1, S2, S1, S2, S5, S1, S1, S1, S5);

Responder a las preguntas siguientes realizando las inferencias.

Explicar el procedimiento e incluir código R y comentarios al resultado en la memoria.

```
library(DescTools);library(PropCIs);library(rcompanion);library(EstimationTools).
```

Estimadores por intervalos.

Dada la muestra de datos de la variable

Data.Carbohydrate:

Sample1 (0.691666, 0, 0.3280065, 0.6374219, 0.6126026, 0, 0.7169533, 0.6157937, 0.690012, 0.5554588, 0.6219643,

0.7177884, 0.6931629, 0.5971366, 0.629137, 0.7084673, 0, 0.7186016, 0.5481317, 0.7437346);

Dada la muestra de datos pareada con Sample1 de la variable

Data.Sugar.Total:

Sample2 (S2, S1, S1, S1, S1, S1, S5, S1, S2, S1, S1, S1, S2, S1, S2, S5, S1, S1, S1, S5);

Responder a las preguntas siguientes realizando las inferencias.

Explicar el procedimiento e incluir código R y comentarios al resultado en la memoria.

```
library(DescTools);library(PropCIs);library(rcompanion);library(EstimationTools).
```

```
library(boot);bootR <- 1000;set.seed(2023); boot.ci(...,conf = 0.95,type=c("norm")).
```

2.1 Estimación puntual

2.1.1 Tenemos que utilizar una distribución normal, utilizando la función `fitdistr()`; para ajustar la muestra Sample1, para calcular la media y la desviación típica, relacionado con Data.Carbohydrate. Con ella calculamos la media (`esS1$estimate[1]`) y la desviación típica (`esS1$estimate[2]`).


```
> esS1 <- fitdistr(Sample1, "normal")
```

```
> esS1$estimate[1]
```

```
mean
```

```
0.541302
```

```
> #52
```

```
> esS1$estimate[2]
```

```
sd
```

```
0.2441435
```

2.1.2 Ahora procedemos a hallar lo mismo, pero esta vez condicionado a `Data.Sugar.Total == S1`, suponiendo siempre una distribución normal.

```
> datoCond <- datos[datos$Sugar.Total == "S1",  
"Carbohydrate"]
```

```
> ajusteDato <- fitdistr(datoCond, "normal")
```

```
> ajusteDato
```

```
mean      sd
```

```
0.45791585 0.26695602
```

Mediante la función `datoCond()`; podemos hallar otra muestra que viene de `Sample1`, con la condición

correspondiente. Después volvemos a ajustar el dato de la misma manera y hallamos la media y la desviación típica.

2.1.3 Por último, volveremos a hacer lo mismo, solo que esta vez estará condicionada por `Data.Sugar.Total==No S1`, sacando nuevamente la media y la desviación típica.

```
> datoCond2 <- datos[datos$Sugar.Total != "S1",  
"Carbohydrate"]  
> ajusteDato2 <- fitdistr(datoCond2, "normal")  
> ajusteDato2$estimate[1]  
mean  
0.6961619  
> ajusteDato2$estimate[2]  
sd  
0.03252742
```

Acorde a los tres subapartados anteriores, se refleja la media de `Data.Carbohydrate` con `Sugar.Total==S1` es mayor que las que tienen `Sugar.Total==NoS1`. Sin embargo, la desviación de los que tienen

Sugar.Total==NoS1 es menor que las que tienen Sugar.Total==S1. Eso indica que los valores que tienen Data.Carbohydrate con Sugar.Total==S1 se dispersan más de la media, y por lo tanto, existe una mayor dispersión para la muestra Sample1.

2.2 Estimación por intervalos

2.2.1 Para ello, estimamos los valores inferior y superior con un índice de confianza del 95% para la media de Sample1 con distribución normal y desviación típica conocida:

```
> ajusteIc <- fitdistr(Sample1, "normal")  
> sigma <- ajusteIc$estimate[2] # Reemplazar con el valor  
conocido si es diferente  
> n <- length(Sample1)  
> media_muestral <- mean(Sample1)  
> z <- qnorm(0.975) # 1.96 para IC 95%  
> inf <- media_muestral - z * (sigma/sqrt(n))  
> inf  
sd  
0.4343033  
> sup <- media_muestral + z * (sigma/sqrt(n))  
> sup
```

sd

0.6483006

El valor inferior sería 0.4343033, y el valor superior sería 0.6483006

Y volvemos a comprobar lo bueno que sería la distribución con el p-valor.

```
sigma <- sd(Sample1)
```

```
desvConocida <- z.test(datos$Carbohydrate,  
mean(datos$Carbohydrate), sd(datos$Carbohydrate),  
conf.level =
```

```
0.95)
```

```
desvConocida
```

```
desvConocida$p.value
```

```
> desvConocida$p.value
```

```
[1] 1
```

Y la media sería:

```
> media <- desvConocida$estimate
```

```
> media
```

```
mean of datos$Carbohydrate
```

```
0.541302
```

2.2.2 Después, estimamos los valores inferior y superior con un índice de confianza del 95% para la media de Sample1 con distribución normal y desviación típica desconocida. Recordemos de que para desviaciones típicas desconocidas utilizamos el `t.test()` en vez del `z.test()`. Además, estimaremos el p-value y la media correspondiente. (Esta es otra forma de resolver el apartado anterior, solo que con `sd()` desconocida):

```
> desvNoConocida <- t.test(datos$Carbohydrate, conf.level  
= 0.95)
```

```
> inf <- desvNoConocida$conf.int[1]
```

```
> inf
```

```
[1] 0.4240709
```

```
> sup <- desvNoConocida$conf.int[2]
```

```
> sup
```

```
[1] 0.658533
```

```
> desvNoConocida$p.value
```

```
[1] 9.09614e-09
```

```
> #64
```

```
> desvNoConocida$estimate
```

```
mean of x
```

```
0.541302
```

Aclaración: `desvNoConocida$estimate` = media estimada de `Sample1`.

2.2.3 Ahora estimamos los valores inferior y superior de un índice de confianza de 95% para la varianza de `Sample1` con una distribución normal. Utilizamos la estimación de IC para muestras pequeñas:

```
> alpha = 1-0.95
```

```
> length(Sample1) #Eso es que la muestra es de tamaño 20
```

```
[1] 20
```

```
> varInf <- ((n-1)*var(Sample1)/qchisq(1-alpha/2, n-1))
```

```
> varInf
```

```
[1] 0.03628727
```

```
> #66
```

```
> varSup <- ((n-1)*var(Sample1)/qchisq(alpha/2, n-1))
```

```
> varSup
```

```
[1] 0.1338482
```

Y la varianza sería:

```
> var(Sample1)
```

```
[1] 0.06274322
```

2.2.4 Ahora se calcula el valor inferior y superior para la proporción de Sample2, junto con el p-value y la media estimada, sabiendo de que I.C. = 95%:

```
> dataFrame <-  
data.frame(Data.Carbohydrate=Sample1,Data.Sugar.Total=  
Sample2)  
  
> tabla_S1 <- table(dataFrame$Data.Sugar.Total == "S1")  
  
> test <- binom.test(tabla_S1[1],20,0.5,alternative =  
"two.sided",conf.level = 0.95)  
  
> print(test)
```

Exact binomial test

data: tabla_S1[1] and 20

number of successes = 7, number of trials = 20, **p-value =
0.2632**

alternative hypothesis: true probability of success is not
equal to 0.5

95 percent confidence interval:

0.1539092 0.5921885

sample estimates:

probability of success (media)

0.35

2.2.5 Aquí estimamos el intervalo de confianza para la diferencia de medias, al nivel de confianza del 95%. Utilizamos un mayor número de réplicas(1000) para el Bootstrap para mayor precisión. La función `boot.ci` calcula el intervalo de confianza del 95% y `boot()` para hallar la media. Siempre bajo la hipótesis de una distribución normal

```
bootR <- 1000
```

```
statistic <- function(data, i){  
  mean(data[i])  
}
```

```
set.seed(2023) #Para reproducir el resultado
```

```
Sample1.boot <- boot(data = Sample1, statistic =  
statistic,bootR)
```

```
boot.ci(Sample1.boot,conf = 0.95,type=c("norm"))
```

BOOTSTRAP CONFIDENCE INTERVAL
CALCULATIONS

Based on 1000 bootstrap replicates

CALL :

```
boot.ci(boot.out = Sample1.boot, conf = 0.95, type =  
c("norm"))
```

Intervals :

Level Normal

95% (0.4300, 0.6487)

Calculations and Intervals on Original Scale

Y para calcular la media: que es 0.541302

Sample1.boot

Call:

boot(data = Sample1, statistic = statistic, R = bootR)

Bootstrap Statistics :

original bias std. error

t1* **0.541302** 0.001977545 0.05578492

2.2.6 Aquí estimamos el intervalo de confianza para las varianzas de Sample1 con Bootstrap, al nivel de confianza del 95%. Siempre bajo la hipótesis de una distribución normal.

```
bootR <- 1000
```

```
statistic <- function(data, i){
```

```
var(data[i]) #En este caso, la varianza
```

```
}
```

```
set.seed(2023)
```

```
Sample1.boot <- boot(data = Sample1, statistic =  
statistic,bootR)
```

```
Sample1.boot
```

```
boot.ci(Sample1.boot,conf = 0.95,type=c("norm"))
```

BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS

Based on 1000 bootstrap replicates

CALL :

```
boot.ci(boot.out = Sample1.boot, conf = 0.95, type =  
c("norm"))
```

Bootstrap Statistics : Aquí la varianza

original bias std. error

t1* **0.06274322** -0.004033139 0.02308194

Intervals : (con valor superior e inferior)

Level Normal

95% (0.0215, 0.1120)

Calculations and Intervals on Original Scale

2.2.6 Aquí estimamos el intervalo de confianza para la diferencia de proporciones de S1 en Sample2 con Bootstrap, al nivel de confianza del 95%. Siempre bajo la hipótesis de una distribución normal. La

proporción se calcula con la fórmula de (numero de S1) / longitudDeMuestra.

```
bootR <- 1000
```

```
proporcion <- function(data, i){  
  return(sum(data[i] == "S1")/length(i))  
}
```

```
set.seed(2023)
```

```
Sample2.boot <- boot(data = Sample2, statistic =  
proporcion, R = bootR)
```

```
Sample2.boot
```

```
boot.ci(Sample2.boot,conf = 0.95,type=c("norm"))
```

Call:

```
boot(data = Sample2, statistic = proporcion, R = bootR)
```

Bootstrap Statistics :

original bias std. error (Aquí la proporción estimada)

```
t1* 0.65 -0.0065 0.1102039
```

```
> boot.ci(Sample2.boot,conf = 0.95,type=c("norm"))
```

BOOTSTRAP CONFIDENCE INTERVAL
CALCULATIONS

Based on 1000 bootstrap replicates

CALL :

```
boot.ci(boot.out = Sample2.boot, conf = 0.95, type =  
c("norm"))
```

Intervals :

Level Normal

95% (0.4405, 0.8725)

Calculations and Intervals on Original Scale

2.2.7 Aquí estimamos el intervalo de confianza para la diferencia de medias de Sample1 condicionado a S1 y a NoS1, con una distribución normal y desviación típica desconocida, al nivel de confianza del 95%. Siempre bajo la hipótesis de una distribución normal

```
dataFrame <-
```

```
data.frame(Data.Carbohydrate=Sample1,Data.Sugar.Total=  
Sample2)
```

```
data1 = dataFrame[dataFrame$Data.Sugar.Total=="S1",]
```

```
data2 = dataFrame[dataFrame$Data.Sugar.Total!="S1",]
```

```
t.test_result<-t.test( x = data1$Data.Carbohydrate, y =  
data2$Data.Carbohydrate, conf.level = 0.95)
```

```
> t.test_result$conf.int[1] #Valor inferior de IC95
```

```
[1] -0.4075908
```

```
> t.test_result$conf.int[2] #Valor superior de IC95
```

```
[1] -0.06890125
```

```
> t.test_result$p.value #p-valor de t.test()
[1] 0.0095799
> dif_medias_est<-t.test_result$estimate[1]-
t.test_result$estimate[2] #diferencia de medias
> dif_medias_est
mean of x
-0.238246.
```

2.2.7 Aquí estimamos el intervalo de confianza para la diferencia de medias de Sample1 condicionado a S1 y a NoS1, con una distribución normal y desviación típica conocida, al nivel de confianza del 95%. Siempre bajo la hipótesis de una distribución normal. Claro, aquí en vez de utilizar t.test() utilizamos z.test():

```
sigx <- sd(data1$Data.Carbohydrate)
sigy <- sd(data2$Data.Carbohydrate)
z.test_result <- z.test( x = data1$Data.Carbohydrate, y =
data2$Data.Carbohydrate,
'two.sided', sigma.x = sigx, sigma.y = sigy, conf.level =
0.95)
> z.test_result$conf.int[1] #Valor inferior
[1] -0.3915138
```

```
> z.test_result$conf.int[2] #Valor superior
```

```
[1] -0.08497819
```

```
> z.test_result$p.value #p.valor
```

```
[1] 0.00231406
```

```
> dif_medias<-z.test_result$estimate[1]-
```

```
z.test_result$estimate[2]
```

```
> dif_medias #diferencia de medias
```

```
mean of x
```

```
-0.238246
```

2.2.8 Por último, calculamos el ratio de varianzas de Sample1/S1 y Sample1/NoS1 con una distribución normal y un nivel de confianza del 95%, junto con valores superiores, inferiores y el p.value():

```
dataFrame <-
```

```
data.frame(Data.Carbohydrate=Sample1,Data.Sugar.Total=Sample2)
```

```
data1 = dataFrame[dataFrame$Data.Sugar.Total=="S1",]
```

```
data2 = dataFrame[dataFrame$Data.Sugar.Total!="S1",]
```

```
>
```

```
var.test(data1$Data.Carbohydrate,data2$Data.Carbohydrate)
```

F test to compare two variances

data: data1\$Data.Carbohydrate and

data2\$Data.Carbohydrate

#Aquí tenemos la ratio estimado de varianzas(F) y p-value
 $F = 62.545$, num df = 12, denom df = 6, p-value = $5.423e-05$

alternative hypothesis: true ratio of variances is not equal to 1

95 percent confidence interval: # Y aquí los valores inferior y superior

11.65534 233.18755

sample estimates:

ratio of variances

62.54541

3. Estimación Bayesiana

Def: La estimación clásica bayesiana, es otro método para estimar parámetros de un modelo con información previa (distribución a priori) y actualiza esta creencia con los datos (teorema de Bayes) para una distribución a posteriori.

Enunciado:

Estimación Bayesiana.

Dada la muestra de datos de la variable

Data.Carbohydrate:

Sample1 (0.691666, 0, 0.3280065, 0.6374219, 0.6126026,
0, 0.7169533, 0.6157937, 0.690012, 0.5554588, 0.6219643,
0.7177884, 0.6931629, 0.5971366, 0.629137, 0.7084673, 0,
0.7186016, 0.5481317, 0.7437346);

Dada la muestra de datos pareada con Sample1 de la
variable

Data.Sugar.Total:

Sample2 (S2, S1, S1, S1, S1, S1, S5, S1, S2, S1, S1, S1, S2,
S1, S2, S5, S1, S1, S1, S5);

Responder a las preguntas siguientes realizando las
inferencias.

Explicar el procedimiento e incluir código R y comentarios
al resultado en la memoria.

Con la estimación bayesiana, podemos hallar la
proporción muestral de Data.Sugar.Total!=S1 (esto es
S2), sacando el cociente entre la cantidad de
elementos no iguales a S1 entre la cantidad de la
muestra S2 por ejemplo:

```
proporcion <-  
length(Sample2[Sample2!="S1"])/length(Sample2)
```


proporción

[1] 0.35

Para hallar el alfa y beta a posteriori, donde el alfa a priori a tomar será 5, y beta a priori será 10, realizamos dos operaciones de suma: Una para el alfa a posteriori, realizándola entre el alfa a priori y la cantidad condicionada con Data.Sugar.Total != S1, y otra para el beta a posteriori, se suma beta a priori mas la diferencia entre el tamaño de la muestra y la cantidad condicionada con Data.Sugar.Total !=S1:

```
> prior.alfa <- 5
```

```
> prior.beta <- 10
```

```
> sample.proports <- table(Sample2)/length(Sample2)
#Proporciones de cada variable
```

```
> samplePe <- 1 - sample.proports["S1"] ## noS1
```

```
> sampleSZ <- length(Sample2) #Tamaño de Sample2
```

```
> r <- samplePe * sampleSZ ## ok
```

```
> n <- sampleSZ
```

```
> alfa.post <- (prior.alfa - 1) + r + 1 #Calculamos alfa a posteriori
```

```
> alfa.post
```

```
S1
```

12

```
> beta.post <- (prior.beta - 1) + n - r + 1 #Calculamos beta  
a posteriori
```

```
> beta.post
```

S1

23

Y con los valores de alfa y beta a posteriori, ya podremos hallar el intervalo con nivel de confianza al 95% para la proporción de Data.Sugar.Total != S1

```
> ic_lower <- qbeta(0.025, alfa.post, beta.post) # Límite  
inferior
```

```
> ic_lower
```

```
[1] 0.1974586
```

```
> ic_upper <- qbeta(0.975, alfa.post, beta.post) # Límite  
superior (valor pedido)
```

```
> ic_upper
```

```
[1] 0.5052653
```

Ahora, calculamos la media posterior con la fórmula bayesiana, y la precisión, con la muestra de Sample1 condicionada a NoS1 de Sample2 (log_noS1)

```
> # Parámetros iniciales (distribución previa)
```

```
> mu0 <- 1.820 # Media previa
```

```

> sd0 <- 0.75    # Desviación estándar previa (¡convertir a
varianza!)

> # Datos (asegúrate de que Sample1 y Sample2 estén
definidos)

> tabla <- data.frame(Sample1, Sample2)

> # 1. Eliminar NA (si existen)

> tabla <- na.omit(tabla)

> # 2. Filtrar registros donde Sample2 NO sea "S1"

> log_noS1 <- tabla[tbl$Sample2 != "S1", ]

> # 3. Calcular estadísticas muestrales

> n <- length(log_noS1$Sample1) # Tamaño muestral

> x <- mean(log_noS1$Sample1)    # Media muestral

> s2 <- var(log_noS1$Sample1)    # Varianza muestral

> # 4. Calcular n0 (usando varianza previa sd0^2, no sd0)

> n0 <- s2 / (sd0^2) # ¡Corrección clave!

> # 5. Media posterior (fórmula bayesiana)

> post_media <- (n * x + n0 * mu0) / (n + n0)

> # Resultado final

> print(post_media)

[1] 0.6965141

> sample.mean <- mean(log_noS1$Sample1) #Media de
Sample 1

```

```
> sample.sd <- sd(log_noS1$Sample1) #Desviación típica
Sample1
> sampleSZby <- length(log_noS1$Sample1) #Tamaño
Sample1
> vero.sd <- sample.sd / (sampleSZby) ^ 0.5 #Cálculo de
error estándar
> prec <- (1 / sd0^2) + (1 / vero.sd^2) #Hallamos la
precision
> prec
[1] 5672.677
```

4. Contrastes

Def: Los contrastes de hipótesis se dividen en 2, paramétricos y no paramétricos:

- Paramétricos: Pruebas que asumen una distribución específica de datos, y contrastan hipótesis sobre sus parámetros

- No paramétricos: Pruebas que no asumen una distribución específica de datos. Es buena para datos no normales.

Enunciados:

Enunciado para contrastes paramétricos en poblaciones normales con una y dos muestras (preguntas con código q41__).

Dada la muestra de datos de la variable

Data.Carbohydrate:

Sample1 (0.691666, 0, 0.3280065, 0.6374219, 0.6126026, 0, 0.7169533, 0.6157937, 0.690012, 0.5554588, 0.6219643, 0.7177884, 0.6931629, 0.5971366, 0.629137, 0.7084673, 0, 0.7186016, 0.5481317, 0.7437346);

Dada la muestra de datos no pareada con Sample1 de la variable

Data.Carbohydrate:

Sample2 (0.7128445, 0.5576923, 0.6409975, 0.3488032, 0.5546417, 0.4341528, 0.7122657, 0.6057349, 0, 0.03590088, 0.6294505, 0.5384865, 0.6066112, 0.6652499, 0.6834918, 0.5449552, 0.2995622, 0.4372671, 0, 0.6092872);

Responder a las preguntas siguientes realizando las inferencias.

Explicar el procedimiento e incluir código R y comentarios al resultado en la memoria.

(quantile(), t.test(), var.test()).

Enunciado para contrastes no paramétricos en poblaciones normales con una y dos muestras (preguntas con código q42__).

En adición a las muestras del enunciado para contrastes paramétricas, se considera la muestra de datos no pareada con Sample1 de la variable

Data.Sugar.Total:

Sample3 (S5, S1, S1, S1, S2, S2, S5, S2, S1, S1, S2, S1, S3, S2, S2, S2, S1, S2, S1, S2);

Responder a las preguntas siguientes realizando las inferencias.

Explicar el procedimiento e incluir código R y comentarios al resultado en la memoria.

(nortest::pearson.test(), stats::chisq.test(), lmtest::dwtest(), stats::wilcox.test()).

Enunciado para contrastes no paramétricos en poblaciones normales con una y dos muestras (preguntas con código q42__).

Considera la muestra de datos de las variables

Data.Fat.Total.Lipid, Data.Vitamins.Vitamin.B12 y

Data.Major.Minerals.Magnesium:

Data.Fat.Total.Lipid (1.534714, 2.652537, 1.663926, 1.324419, 1.302913, 2.40243, 2.277267, 0.5877867, 1.458615, 2.005526, 2.712706, 3.07223, 1.483875, 1.867176, 0.3001046, 3.233961, 2.551786, 3.08191, 1.986504, 0);

Data.Vitamins.Vitamin.B12 (0.05826891, 1.238374, 0.04879016, 0, 0, 1.280934, 0.14842, 0, 0, 0.09531018, 0,

0, 0.05826891, 0.05826891, 0, 0.3920421, 0.3220835, 0, 0.2468601, 0);

Data.Major.Minerals.Magnesium (3.73767, 3.178054, 1.94591, 2.564949, 3.583519, 3.218876, 2.833213, 3.526361, 3.713572, 2.890372, 3.135494, 4.488636, 2.995732, 2.70805, 3.258097, 4.248495, 3.091042, 4.49981, 2.397895, 5.389072);

Sabemos además que $\text{Data.Major.Minerals.Magnesium} \sim \text{Data.Fat.Total.Lipid} + \text{Data.Vitamins.Vitamin.B12}$.

4.1 Contrastes paramétricos

Tenemos la muestra `Sample1`, utilizada en apartados anteriores. Estos son sus detalles:

```
> summary(Sample1)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.0000	0.5536	0.6256	0.5413	0.6970	0.7437

```
> #101
```

```
> quantile(Sample1)
```

0%	25%	50%	75%	100%
0.0000000	0.5536270	0.6255506	0.6969890	0.7437346

Donde `quantile` para 25% es el cuartil 1, y para el 75% es el cuartil 3.

Ahora bien, queremos contrastar si la media es mayor que Q1 (Hipótesis inicial), con índice de confianza del 95%. Para ello, realizaremos una hipótesis, y ya que la varianza es desconocida, utilizaremos `t.test()`.

```
t.test(Sample1, mu = quantile(Sample1, probs = 0.25),  
alternative = "less") #Alternative less por la hipótesis  
alternativa, si no se cumple la hipótesis inicial, se cumple lo  
contrario
```

One Sample t-test

data: Sample1

t = -0.22005, df = 19, p-value = 0.4141

alternative hypothesis: true mean is less than 0.553627

95 percent confidence interval:

-Inf 0.6381513

sample estimates:

mean of x

0.541302

Con esto, hallamos el estadístico t, los grados de libertad n (df) y el p-value.

Ahora probamos haciendo lo contrario, tomando la hipótesis inicial anterior como alternativa, y la hipótesis alternativa anterior como inicial.

```
> test <- t.test(Sample1, mu = quantile(Sample1, probs =  
0.25), alternative = "greater")
```

```
> test
```

One Sample t-test

data: Sample1

t = -0.22005, df = 19, p-value = 0.5859

alternative hypothesis: true mean is greater than 0.553627

95 percent confidence interval:

0.4444526 Inf

sample estimates:

mean of x

0.541302

```
> test$statistic #Para hallar el valor completo de t
```

t

-0.2200497

Y consecuentemente, lo probamos haciendo con alternativa “two.sided”, si solo queremos tener la alternativa de que la media difiere de Q1.

```
> test <- t.test(Sample1, mu = quantile(Sample1, probs =  
0.25), alternative = "two.sided")
```

```
> test
```

One Sample t-test

data: Sample1

t = -0.22005, df = 19, p-value = 0.8282

alternative hypothesis: true mean is not equal to 0.553627

95 percent confidence interval:

0.4240709 0.6585330

sample estimates:

mean of x

0.541302

Muy bien, ahora vamos a repetir todo el proceso con el cuartil 3(probs de 0.75) en vez del cuartil 1, donde la hipótesis iniciales son: $\text{media} > \text{cuartil3}$, $\text{media} \leq \text{cuartil 3}$, $\text{media} = \text{cuartil 3}$. Vamos cambiando el valor de la hipótesis alternativa en las funciones

Media $>$ cuartil3:

```
> test <- t.test(Sample1, mu = quantile(Sample1, probs =  
0.75), alternative = "less")
```

```
> test
```

One Sample t-test

data: Sample1

$t = -2.7796$, $df = 19$, $p\text{-value} = 0.005971$

alternative hypothesis: true mean is less than 0.696989

95 percent confidence interval:

-Inf 0.6381513

sample estimates:

mean of x

0.541302

Media <= cuartil 3:

```
> test <- t.test(Sample1, mu = quantile(Sample1, probs =  
0.75), alternative = "greater")
```

```
> test
```

One Sample t-test

data: Sample1

$t = -2.7796$, $df = 19$, $p\text{-value} = 0.994$

alternative hypothesis: true mean is greater than 0.696989

95 percent confidence interval:

0.4444526 Inf

sample estimates:

mean of x

0.541302

Media = cuartil3:

```
> test <- t.test(Sample1, mu = quantile(Sample1, probs =  
0.75), alternative = "two.sided")
```

```
> test
```

One Sample t-test

data: Sample1

t = -2.7796, df = 19, p-value = 0.01194

alternative hypothesis: true mean is not equal to 0.696989

95 percent confidence interval:

0.4240709 0.6585330

sample estimates:

mean of x

0.541302

Ahora hipotetizamos si la varianza es mayor que 1.

Calculamos el estadístico d (x-squared), los grados de libertad y el p-value:

```
> var_test
```

One Sample Chi-Square test on variance

data: Sample1

X-squared = 1.1921, df = 19, p-value = 3.778e-09

alternative hypothesis: true variance is less than 1

95 percent confidence interval:

0.0000000 0.1178333

sample estimates:

variance of x

0.06274322

Otra prueba para realizar es contrastar con la hipótesis inicial, donde la media de la muestra 1 es igual que la media e la muestra 2, con nivel de confianza al 95%. Se saca el estadístico t, los grados de libertad n y el p-value:

```
> Sample2<- c(0.7128445, 0.5576923, 0.6409975,  
0.3488032, 0.5546417, 0.4341528, 0.7122657, 0.6057349,  
0, 0.03590088, 0.6294505, 0.5384865, 0.6066112,  
0.6652499, 0.6834918, 0.5449552, 0.2995622, 0.4372671,  
0, 0.6092872)
```

```
> t_test <- t.test(Sample1, Sample2, alternative =  
"two.sided", var.equal = TRUE, conf.level = 0.95)
```

```
> t_test
```

Two Sample t-test

data: Sample1 and Sample2

t = 0.79355, df = 38, p-value = 0.4324

alternative hypothesis: true difference in means is not equal to 0

95 percent confidence interval:

-0.09373325 0.21459766

sample estimates:

mean of x mean of y

0.5413020 0.4808698

Y ahora, contrastamos con la hipótesis inicial de que las varianzas de la muestra 1 y 2 son iguales, con el mismo coeficiente de confianza de 95%.

Calculamos el estadístico t, los grados de libertad n (nota, siempre fue la longitud de la muestra – 1) y el p-value con el f.test():

```
> f_test <- var.test(Sample1, Sample2, alternative =  
"two.sided")
```

```
> f_test
```

F test to compare two variances

data: Sample1 and Sample2

**F = 1.1784, num df = 19, denom df = 19, p-value =
0.7242**

alternative hypothesis: true ratio of variances is not equal to
1

95 percent confidence interval:

0.4664197 2.9771357

sample estimates:

ratio of variances

1.178386

4.1 Contrastes no paramétricos

Con los contrastes no paramétricos, podemos tomar la muestra `Sample1`, y calcular el estadístico Pearson chi-square. A la vez, identificar el número de clases utilizadas en el test:

```
> pearson_test <- nortest::pearson.test(Sample1)
```

```
> pearson_test
```

Pearson chi-square normality test

data: `Sample1` (P = estadístico Pearson)

P = 19.9, p-value = 0.0005226

```
> pearson_test$n.classes
```

```
[1] 7
```

Para la hipótesis compuesta con normalidad, calculamos la prueba de chi-cuadrado de Pearson:

```

> # Prueba de normalidad usando la prueba de chi-cuadrado

> # estimación de los parámetros de la normalidad

> mu <- mean(Sample1)
> sigma <- sd(Sample1)
>
> # Intervalos
> breaks <- seq(min(Sample1), max(Sample1), length.out = 6) # Dividir en 5 intervalos
>
> # Frecuencia observada
> observed <- table(cut(Sample1, breaks = breaks, include.lowest = TRUE))
>
> # Frecuencia esperada bajo una distribución normal
> expected <- diff(pnorm(breaks, mean = mu, sd = sigma)) * length(Sample1)
>
> # prueba chi-cuadrado
> chi_square_test <- chisq.test(observed, p = expected / sum(expected), rescale.p = TRUE)
# p-value
> chi_square_test$p.value
[1] 3.331007e-05

```


Ahora, en vez de utilizar chi-cuadrado, realizamos la prueba con la teoría de kolmogorov-smirnov para calcular el estadístico de Pearson chi-cuadrado y el p-value:

```
> out <- ks.test(Sample1, "pnorm", mean(Sample1),  
sd(Sample1))
```

```
> print(out)
```

Asymptotic one-sample Kolmogorov-Smirnov test

data: Sample1

D = 0.31088, p-value = 0.04189

alternative hypothesis: two-sided

```
> "est_pearson" <- out$statistic
```

```
> "pearsonPvalue" <- out$p.value
```

```
> est_pearson
```

D

0.3108762

```
> pearsonPvalue
```

```
[1] 0.04189359
```

La siguiente prueba consiste en probar con tablas de contingencia de chi-cuadrado con la homogeneidad de la muestra 3 (Sample3) y la distribución uniforme

discreta ($1/n$, $1/n$). Calculamos el estadístico chi-cuadrado, los grados de libertad y el p-value, viendo si varias muestras provienen de la misma población:

```
> Sample3 = c("S5", "S1", "S1", "S1", "S2", "S2", "S5",  
"S2", "S1", "S1", "S2", "S1", "S3", "S2", "S2", "S2", "S1",  
"S2", "S1", "S2");  
  
> out <- stats::chisq.test(table(Sample3))  
  
> print("stats::chisq.test( table(Sample3))")  
[1] "stats::chisq.test( table(Sample3))"  
  
> print(out)
```

Chi-squared test for given probabilities

data: table(Sample3)

X-squared = 10, df = 3, p-value = 0.01857

Otra de las pruebas que podemos realizar, es utilizar las pruebas de Wilcoxon con Sample1 y 2 en vectores de datos para ver si varias muestras provienen de la misma población:

```
> # Sample1 y Sample2  
> Sample2 <- c(0.7128445, 0.5576923, 0.6  
409975, 0.3488032, 0.5546417, 0.4341528,  
0.7122657, 0.6057349, 0,
```

```

+           0.03590088, 0.6294505, 0.
5384865, 0.6066112, 0.6652499, 0.6834918
, 0.5449552, 0.2995622,
+           0.4372671, 0, 0.6092872)
>
> # Prueba de wilcoxon
> wilcox.test(Sample1, Sample2, alternat
ive = "two.sided")

```

```

      wilcoxon rank sum test with continu
ity correction
data:  Sample1 and Sample2
W = 260, p-value = 0.1072
alternative hypothesis: true location sh
ift is not equal to 0

```

Por último, realizamos pruebas con Durbin-Watson para la autocorrelación de perturbaciones normalmente distribuidas. Calculamos el estadístico dwtest con muestras del enunciado 3 (Data.Fat.Total.Lipid, Data.Vitamins.Vitamin.B12 y Data.Major.Minerals.Magnesium):

```

> Data20.Fat.Total.Lipid <- c(1.534714, 2.652537,
1.663926, 1.324419, 1.302913, 2.40243, 2.277267,
0.5877867, 1.458615, 2.005526, 2.712706, 3.07223,
1.483875, 1.867176, 0.3001046, 3.233961, 2.551786,
3.08191, 1.986504, 0);

> Data20.Vitamins.Vitamin.B12 <- c(0.05826891,
1.238374, 0.04879016, 0, 0, 1.280934, 0.14842, 0, 0,
0.09531018, 0, 0, 0.05826891, 0.05826891, 0, 0.3920421,
0.3220835, 0, 0.2468601, 0);

```

```
> Data20.Major.Minerals.Magnesium <- c(3.73767,
3.178054, 1.94591, 2.564949, 3.583519, 3.218876,
2.833213, 3.526361, 3.713572, 2.890372, 3.135494,
4.488636, 2.995732, 2.70805, 3.258097, 4.248495,
3.091042, 4.49981, 2.397895, 5.389072);

> dwt <- lmtest::dwtest(Data20.Major.Minerals.Magnesium
~ Data20.Fat.Total.Lipid + Data20.Vitamins.Vitamin.B12,
alternative = "two.sided")

> print(dwt)
```

Durbin-Watson test

```
data: Data20.Major.Minerals.Magnesium ~
Data20.Fat.Total.Lipid + Data20.Vitamins.Vitamin.B12
DW = 2.1506, p-value = 0.7502
alternative hypothesis: true autocorrelation is not 0
```

```
> "dwtest" <- dwt$statistic ##-- dwt
```

```
> "dwPvalue" <- dwt$p.value
```

```
> dwtest
```

DW

```
2.150599
```

```
> dwPvalue
```

```
[1] 0.7501886
```

5. Regresión lineal

Def: Es un modelo que predice una variable dependiente como combinación de variables independientes.

Las variables predictoras pueden ser una o varias.

Enunciado:

Enunciado para Regresión lineal en poblaciones normales (preguntas con código q51__).

Dada la muestra de datos de las variables

Data.Carbohydrate:

Sample1 (0.691666, 0, 0.3280065, 0.6374219, 0.6126026, 0, 0.7169533, 0.6157937, 0.690012, 0.5554588, 0.6219643, 0.7177884, 0.6931629, 0.5971366, 0.629137, 0.7084673, 0, 0.7186016, 0.5481317, 0.7437346);

Dada la muestra de datos de la variable

Data.Protein:

Sample1 (0.5180619, 0.6492829, 0.2524786, 0.3581127, 0.3643115, 0.6586576, 0.4204763, 0.3418198, 0.5120559, 0.4466532, 0.3527155, 0.4889995, 0.5138677, 0.4845214, 0.4765505, 0.4982325, 0.6194884, 0.4902204, 0.4710988, 0);

Responder a las preguntas siguientes realizando las inferencias.

Explicar el procedimiento e incluir código R y comentarios al resultado en la memoria.

(quantile(), t.test(), var.test()).

Tenemos una regresión lineal simple de respuesta la muestra Data.Protein y predictora Data.Carbohydrate. Calculamos las variables Beta0 para definir el valor de la variable Data.Protein si la variable predictora es 0, Beta1 para saber cómo cuantifica la variable respuesta por cada unidad que aumenta Data.Carbohydrate y R2 para medir el porcentaje de variabilidad en Data.Protein explicada por Data.Carbohydrate, es decir, te dice si el modelo es útil o le faltan variables clave:

```
modelo <- lm(protein ~ carbohydrate)
> coef(modelo)[1] # Beta0
(Intercept)
  0.6011558
> coef(modelo)[2] # Beta1
carbohydrate
-0.2868556
> summary(modelo)$r.squared # R2 del modelo
[1] 0.2384662
```

Ahora, queremos calcular el p-valor con la hipótesis de $\beta_0 = 0$, de $\beta_1 = 1$ y el estadístico de F del C. de Regresión para probar la hipótesis nula, donde al menos un coeficiente no es cero, y determina si el modelo es significativo o no, donde el modelo al menos tiene un predictor que aporte información relevante:

```
> mod <- lm(datos$Protein ~ datos$Carbohydrate, data = datos)
```

```
> summary_modelo <- summary(modelo)
```

```
> summary_modelo
```

Call:

```
lm(formula = datos$Protein ~ datos$Carbohydrate, data = datos)
```

Residuals:

Min	1Q	Median	3Q	Max
-0.38781	-0.06043	0.03765	0.09411	0.11531

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.60116	0.07175	8.379	1.26e-07 ***
datos\$Carbohydrate	-0.28686	0.12083	-2.374	0.0289 *

Signif. codes:

0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1319 on 18 degrees of freedom

Multiple R-squared: 0.2385, Adjusted R-squared: 0.1962

F-statistic: 5.637 on 1 and 18 DF, p-value: 0.02892

```
> p_beta <-
```

```
summary_modelo$coefficients["(Intercept)","Pr(>|t|)"]
```

```
> p_valor_beta1 <-
```

```
summary_modelo$coefficients["datos$Carbohydrate","Pr(>|t|)"]
```

```
> p_beta
```

```
[1] 1.260398e-07
```

```
> p_valor_beta1
```

```
[1] 0.02891968
```

```
> F_statistic_summary <- summary_modelo$fstatistic[1]
```

```
> F_statistic_summary
```

```
value
```

```
5.636508
```

Y los datos de libertad de regresión se pueden hallar con la diferencia de la muestra predictora entre 1, siendo resultante **19**.

Por último, ya que tenemos Data.Protein y Data.Carbohydrate, queremos hallar la estimación

puntual de la predicción del mínimo, de la media y del máximo, junto con los extremos inferiores y superiores de IC.

Predicción del mínimo:

```
> modelo <- lm(datos$Protein ~ datos$Carbohydrate, data = datos)
```

```
> summary(modelo)
```

Call:

```
lm(formula = datos$Protein ~ datos$Carbohydrate, data = datos)
```

Residuals:

Min	1Q	Median	3Q	Max
-0.38781	-0.06043	0.03765	0.09411	0.11531

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.60116	0.07175	8.379	1.26e-07 ***
datos\$Carbohydrate	-0.28686	0.12083	-2.374	0.0289 *

Signif. codes:

0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1319 on 18 degrees of freedom

Multiple R-squared: 0.2385, Adjusted R-squared: 0.1962

F-statistic: 5.637 on 1 and 18 DF, p-value: 0.02892

```
> new <- data.frame(h = c(min(datos$Carbohydrate),  
mean(datos$Carbohydrate), max(datos$Carbohydrate)))
```

```
> preds <- predict.lm(modelo, new, interval = "confidence",  
level = 0.95, type = "response")
```

```
> preds
```

	fit	lwr	upr
1	0.4027475	0.3299620	0.4755330
2	0.6011558	0.4504194	0.7518921
3	0.5070653	0.4247706	0.5893599

Interpretación: Hemos realizado un análisis del modelo de regresión lineal para Data.Protein y Data.Carbohydrate con `summary(modelo)`. Y hemos utilizado `predict.lm` para hallar las estimaciones puntuales del mínimo, de la media y del máximo.

En la fila 1 se ubica: estimación puntual, extremo inferior, extremo superior del mínimo de la muestra Data.Carbohydrate

En la fila 2 se ubica: estimación puntual, extremo inferior, extremo superior de la media de la muestra Data.Carbohydrate

En la fila 3 se ubica: estimación puntual, extremo inferior, extremo superior del máximo de la muestra Data.Carbohydrate

El orden se definió mediante los valores del dataframe “new”.

6. Anexo: Conjunto de librerías utilizadas

```
install.packages("e1071") #skewness, curtosis
install.packages("MASS") #Análisis y regresión
install.packages("fitdistrplus") #Ajuste de distribuciones
install.packages("DescTools") #Intervalos de confianza
install.packages("PropCIs") #Proporciones
install.packages("rcompanion") #Análisis estadísticos
install.packages("EstimationTools") #Estimación puntual e
interválica
install.packages("boot") #Bootstrap para inferencia
estadística
install.packages("nortest") #Test de normalidad
install.packages("BSDA") #Para análisis con z.test()
install.packages("lmtest") #Para regresiones lineales
library("lmtest")
library("MASS") #
library("fitdistrplus") #
library("e1071")
library("DescTools")
library("PropCIs")
library("rcompanion")
library("EstimationTools")
library("boot")
library("BSDA")
```